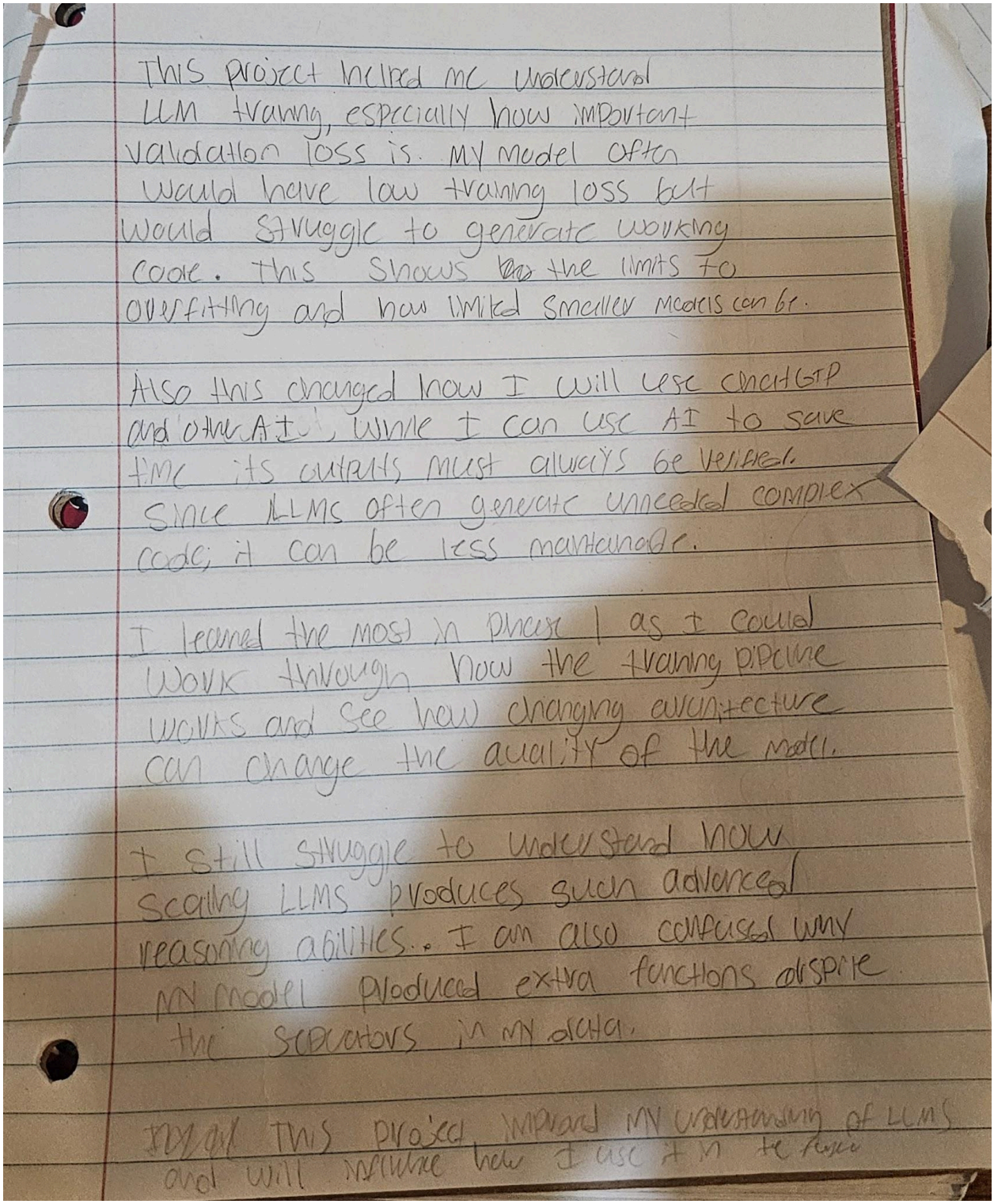
Reflection



This project helped me understand LLM training, especially how important validation loss is. My model often would have low training loss but would struggle to generate working code. This shows the limits to overfitting and how limited smaller models can be.

Also this changed how I will use ChatGTP and other AI's, while I can use AI to save time its outputs must always be verified. Since LLMs often generate unneeded complex code; it can be less maintainable.

I learned the most in phase 1 as I could work through how the training pipeline works and see how changing architecture can change the quality of the model.

I still struggle to understand how scaling LLMS produces such advanced reasoning abilities.. I am also confused why my model produced extra functions despite the separators in my data.

Overall This project improved my understanding of LLMS and will influence how I use it in the future

AI Use Log

Throughout the project the goal was to use AI in ways that would enhance learning and the quality of the project, without taking away the opportunity to learn or get the core understanding of the project. In order to keep this goal I had a few basic policies which determined how I used AI

-To create pipeline functions that would be incredibly time consuming and tedious to build with current knowledge.

This can be seen in my repositories under comments marked **AI USE**. I used AI in order to learn the basics about how to connect Github repositories to Google Colab and how to edit them on the spot. So this included functions such as using the Path library to find files and functions and use them to edit the original Nano GTP.

I also used AI in order to build Pipeline steps that I had little idea on how to build, this includes the tokenizer and how to include the end of text boundary along with the **extract_function** file which pulled functions out of the Python files. With these uses I made sure to study and understand the code.

To verify I made sure I understood the code it was generating which allowed me to catch some misunderstandings it had when generating. I also made sure to run all the code to make sure it actually worked and completed the task as designed. I also made sure to look and understand NanoGTP well to make sure it was not creating duplicate functions that already existed inside of it.

This is where the AI made mistakes, since I did not give it the entire context of the code, it often would try generating code that already existed in different parts of the notebook and often would include functions that were unnecessarily complex. For example it tried creating its own separate transformer implementation using PyTorches nn.MultiheadAttention. This is not necessary because this functionality was already inside of nanoGTP so I threw this out.

I asked ChatGPT to explain just what nanoGPT's architecture is and how it learns for academic purposes so I could understand it as well as I wanted.

For step 2 I used AI to develop the test functions. This was for two reasons,

A: Generating functions is trivial for me and would be time consuming with little academic benefit.

B: I was able to get the model to generate functions with different difficulty levels which improved the validation.

C: It was able to create functions that simulate real software engineering tasks without trying to find a bunch of separate repositories

I made sure I understood all functions and how they would work as it is important to understand how the AI potentially could mess up the outputs. I also made sure they were functions that were feasible and properly tested the real software engineering task of code completion.

For step 3 I had it generate some candidate tests too (that were used in step 4). It tried to generate the same test twice, so I deleted it, it also created some tests that I thought were not great so I rewrote some of them by hand and others I regenerated.

This demonstrated that while AI is powerful too, it must be verified and taken with a grain of salt. If I just used AI without verification it would create a bloated codebase that is both difficult to use, and less functional.

Part 2 inputs and outputs

Python

```python
prompts = [
    # EASY (10)
    """def square(x):
    return x *""",

    """def add(a, b):
    return a +""",

    """def is_even(n):
    return n % 2 ==""",

    """def greet(name):
    return f"Hello, {""",

    """def double(n):
    return 2 *""",

    """def absolute_value(n):
    if n < 0:
        return""",

    """def celsius_to_fahrenheit(c):
    return c * 9 / 5 +""",

    """def uppercase(text):
    return text.""",

    """def last_word(sentence):
    words = sentence.split()
    return""",

    """def remove_spaces(text):
```

```
    return text.replace("""",


    # MODERATE (5)
    """def factorial(n):
    if n <= 1:
        return 1
    return n *""",

    """def count_items(items):
    count = 0
    for item in items:
        count +=""",

    """def find_max(items):
    largest = items[0]
    for item in items[1:]:
""",

    """def contains(items, target):
    for item in items:
        if item == target:
""",

    """def reverse_list(items):
    result = []
    for item in items:
""",


    # HARDER / DOCSTRING (5)
    '''def is_positive(n):
```

```
    """Return True if n is greater than zero, otherwise False."""
    ''',

    '''def multiply(a, b):
    """Return the product of a and b."""
    return a ''',

    '''def first_element(items):
    """Return the first element of a list, or None if empty."""
    ''',

    '''def sum_list(items):
    """Return the sum of all numbers in the list."""
    ''',

    '''def is_empty(items):
    """Return True if the collection is empty, otherwise False."""
    '''
]
```

Outputs

Python

```
============================================================


def square(x):
    return x * y
```

```python
def main() -> None:
    print(f"square square: {square_square_area(2, 3)}")
    print(f"square square: {square_square_area(2, 3


============================================================

def add(a, b):
    return a + b

def main() -> None:
    print(f"a = {a}")
    print(f"a == b -> {a == b}")
    print(f"a == b -> {a ==


============================================================

def is_even(n):
    return n % 2 == 0

def main() -> None:
    print(f"primes below 100: {primes(1000)}")
    print(f"primes below 100: {primes(1000)}")

    print(f"primes below


============================================================

def greet(name):
    return f"Hello, {name} {lastname} {lastname}.")

def main() -> None:
    print(f"Hello {name} {lastname}")
    print(f"Hello {name} {lastname}")
    print(f


============================================================
def double(n):
    return 2 * n

def main() -> None:
    print(f"factorial_iterative(10) = {factorial_iterative(10)}")
```

```
    print(f"factorial_iterative(10) = {factorial_iterative(10


============================================================

def absolute_value(n):
    if n < 0:
        return 0.0
    return 0.0

def main() -> None:
    print(f"factorial_iterative(10) = {factorial_iterative(10)}")
    print(f"factorial_iterative(10


============================================================

def celsius_to_fahrenheit(c):
    return c * 9 / 5 + 32

def celsius_to_fahrenheit(fahrenheit: float, ndigits: int = 2) -> float:
    """
    Convert a given value from Fahrenheit to Kelvin and round it to 2 decimal
places.


============================================================

def uppercase(text):
    return text.replace(" ", "").replace(" ", "").replace(" ", "").replace(" ",
"").replace(" ", "").replace(" ", "").replace(" ", "").replace("-",
"").replace("-", "").replace("-", "").replace("-", ""))


============================================================

def last_word(sentence):
    words = sentence.split()
    return words[0]

def get_word_count(sentence: str) -> int:
    """
    Return the number of words in a given word.

    >>> get_word_count("a")
```

```
============================================================

def remove_spaces(text):
    return text.replace(text)

def __init__(self, text: str, text: Text) -> None:
    self.text = text
    self.text = text
    self.text = Text(text)
    self



============================================================

def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n - 1)

def factorial(n: int) -> int:
    if n < 0:
        return 1
    return factorial(n - 1)

def factorial(n: int


============================================================

def count_items(items):
    count = 0
    for item in items:
        count += count_items(items)
    return count

def count_items_schema(items: list, items: list) -> dict:
    if isinstance(items, dict):
        return {
            'items':
```

```
============================================================

def find_max(items):
    largest = items[0]
    for item in items[1:]:
        largest = max(item, max(item)
    return largest

def solution(n: int = 1000) -> int:
    """
    Returns the sum of all the digits in the factorial of n


============================================================

def contains(items, target):
    for item in items:
        if item == target:
            return
        elif item in seen:
            return set(set(item), set(set(item))
        else:
            raise ValueError(f"Invalid input: {item!r} is not a valid set")


============================================================

def reverse_list(items):
    result = []
    for item in items:
        if item not in items:
            result.append(item)
    return result

def main() -> None:
    items = [1, 2, 3, 4, 5]
    print(f"



============================================================
def is_positive(n):
    """Return True if n is greater than zero, otherwise False."""
     return n < 0.0
    return n == 0.0
```

```
def main() -> None:
    n = int(input("Enter the number of nodes: "))
    nodes = [nodes[0]]


============================================================

def multiply(a, b):
    """Return the product of a and b."""
    return a  # a * b

def main() -> None:
    print(f"product([1, 2, 3], [3, 4]) = {product([1, 2, 3])}")
    print(f"product


============================================================

def first_element(items):
    """Return the first element of a list, or None if empty."""
     first = first
    for item in items:
        if item not in first:
            yield first
        else:
            yield first

def main() -> None:
    print(f"first_element(["


============================================================

def sum_list(items):
    """Return the sum of all numbers in the list."""
     sum_items = sum(items)
    return sum(
        sum(items)
        for item in items
        if sum_items == sum_items
    )

def sum_of_items(items
```

```
============================================================

def is_empty(items):
    """Return True if the collection is empty, otherwise False."""
     return True
    if not items:
        return False

    # Check if the collection is empty, False otherwise.
    if collection == collection:
        return True

    # Check if the collection is empty, return the
```